

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
Facultad de Ingeniería, Matemática y Ciencias Físicas

MANUAL TÉCNICO
EcoRecicla IA

Clasificación de Desechos con Visión Artificial
Proyecto 01 — Curso 045: Inteligencia Artificial

Clasificación Inteligente de Residuos

GUATEMALA, MAYO 2026

ÍNDICE

Cont.	Pág.
INTRODUCCIÓN.....	I
1. OBJETIVO DEL MANUAL	1
2. DESCRIPCIÓN GENERAL DEL SISTEMA	1
2.1 Características Principales	1
2.2 Arquitectura del Sistema.....	2
2.3 Diagrama de Arquitectura del Sistema	2
3. CONFIGURACIÓN Y DESPLIEGUE	3
3.1 Requisitos del Sistema	3
3.2 Instalación Paso a Paso	3
3.3 Comandos Disponibles.....	4
4. ESTRUCTURA DEL PROYECTO.....	4
5. DESCRIPCIÓN DE LOS MÓDULOS PRINCIPALES	6
5.1 src/model.py — Modelo y Mapeo	6
5.2 src/preprocess.py — Preprocesamiento	7
5.3 src/train.py — Entrenamiento	8
5.4 src/evaluate.py — Evaluación.....	8
5.5 src/predict.py — Inferencia	8
6. DATASET Y FLUJO DE DATOS.....	10
6.1 Fuentes del Dataset	10
6.2 Pipeline de Procesamiento de Datos	10
6.3 Distribución del Dataset.....	11
7. INTERFAZ DE USUARIO — APP/APP.PY	11
7.1 Descripción General	11
7.2 Modo Foto Instantánea.....	11
7.3 Modo Video en Tiempo Real	12
7.4 Modo Archivo	12
7.5 Umbral de Duda	12
8. DEPENDENCIAS Y VERSIONES.....	12
9. SEGURIDAD, PRIVACIDAD Y MANTENIMIENTO.....	13
9.1 Privacidad por Diseño.....	13
9.2 Limitaciones de Seguridad	14
9.3 Mantenimiento	14
9.3.2 Control de versiones del modelo	15
9.3.3 Actualización de dependencias	15

9.4 Resolución de Problemas Comunes (Troubleshooting)	15
9.5 Procedimiento de Validación Completa (Checklist).....	16
10. FUTURAS MEJORAS Y ESCALABILIDAD.....	16
10.1 Opción A — Streamlit Community Cloud	17
10.3 Comparativa de Opciones	17
10.4 Nota sobre despliegue actual	17
11. FUTURAS MEJORAS Y ESCALABILIDAD (AMPLIADO).....	18
11.1 Mejoras Priorizadas:	18
CONCLUSIÓN.....	19

INTRODUCCIÓN

El presente Manual Técnico documenta la arquitectura, configuración, instalación y despliegue del sistema EcoRecicla IA, desarrollado como Proyecto 01 del Curso 045 — Inteligencia Artificial de la Universidad Mariano Gálvez de Guatemala.

Este documento está dirigido a desarrolladores y evaluadores técnicos que necesiten comprender la estructura interna del sistema, replicar el entorno de desarrollo, ejecutar el entrenamiento del modelo o desplegar la aplicación. No requiere conocimientos previos del proyecto, pero sí familiaridad con Python 3.10+, conceptos básicos de machine learning y manejo de terminal.

El sistema clasifica residuos sólidos en 10 categorías mediante Transfer Learning sobre MobileNetV2, y los presenta al usuario con la recomendación del recipiente correcto según el Acuerdo Gubernativo 164-2021 del MARN Guatemala. La interfaz web fue construida con Streamlit e integra tres modos de entrada: foto instantánea, video en tiempo real vía WebRTC y análisis de archivos.

1. OBJETIVO DEL MANUAL

Proporcionar a desarrolladores, evaluadores técnicos y personal de mantenimiento toda la información necesaria para:

- Comprender la arquitectura completa del sistema EcoRecicla IA.
- Replicar el entorno de desarrollo de forma reproducible.
- Ejecutar el pipeline de entrenamiento del modelo desde cero.
- Lanzar y operar la aplicación Streamlit con los tres modos de entrada.
- Agregar nuevas clases de residuos al modelo sin modificar la arquitectura base.
- Diagnosticar y resolver problemas técnicos comunes.

2. DESCRIPCIÓN GENERAL DEL SISTEMA

EcoRecicla IA es una aplicación web de inteligencia artificial que permite clasificar residuos sólidos a partir de imágenes o video en tiempo real. El sistema integra un modelo de visión artificial (MobileNetV2 con Transfer Learning) con una interfaz web interactiva (Streamlit) para ofrecer clasificación instantánea con recomendación de recipiente según la normativa guatemalteca AGG 164-2021.

2.1 Características Principales

CARACTERÍSTICA	DESCRIPCIÓN
Modelo de clasificación	MobileNetV2 con Transfer Learning — 10 clases de residuos sólidos.
Precisión (prueba)	83.17% accuracy global · 94.23% Top-2 · 78.12% F1-Macro.
Modos de entrada	Foto instantánea, video en tiempo real (WebRTC) y archivo (imagen/video).
Normativa aplicada	AGG 164-2021 MARN Guatemala — 3 categorías: blanco, verde, negro.
Umbral de duda adaptativo	CLASS_THRESHOLDS por clase (45%-70%) — alerta cuando la confianza es baja.

Tecnología web	Streamlit + streamlit-webrtc — sin HTML/CSS/JS adicional.
Entorno	Python 3.10+ · Compatible con Windows, Linux y macOS.
Privacidad	Sin almacenamiento de imágenes — procesamiento en memoria.

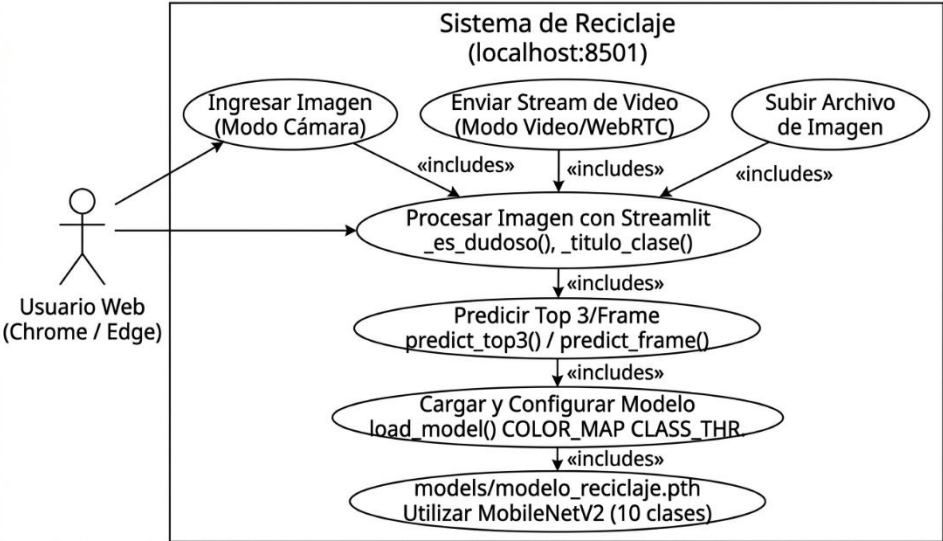
2.2 Arquitectura del Sistema

EcoRecicla IA sigue una arquitectura de tres capas desacopladas:

CAPA	COMPONENTE	RESPONSABILIDAD
Datos	data/ + src/split_dataset.py	Dataset organizado por clase, splits estratificados 70/15/15.
Modelo	src/ (model, preprocess, train, evaluate, predict)	Pipeline de ML: preprocesamiento, entrenamiento, evaluación e inferencia.
Presentación	app/app.py + streamlit-webrtc	Interfaz web con 3 modos de entrada, renderizado de resultados y umbral de duda.

2.3 Diagrama de Arquitectura del Sistema

EcoRecicla IA sigue una arquitectura cliente-servidor ejecutándose completamente en el entorno local del usuario. El siguiente diagrama representa visualmente la interacción entre los componentes:



3. CONFIGURACIÓN Y DESPLIEGUE

3.1 Requisitos del Sistema

REQUISITO	ESPECIFICACIÓN MÍNIMA
Sistema operativo	Windows 10+, Ubuntu 20.04+ o macOS 12+
Python	3.10 o superior (recomendado: 3.11)
RAM	8 GB mínimo (16 GB recomendado para entrenamiento)
Almacenamiento	5 GB libres (dataset + modelo + dependencias)
GPU (opcional)	CUDA compatible para acelerar el entrenamiento. No requerida para inferencia.
Cámara web	Cualquier cámara web USB o integrada para los modos foto y video.
Navegador	Google Chrome, Firefox o Edge — versiones actualizadas para WebRTC.
Conexión a internet	Requerida para el modo video en tiempo real (WebRTC).

3.2 Instalación Paso a Paso

Paso 1 — Clonar el repositorio

```
git clone https://github.com/Javiii3er/reciclaje-ia-grupo4.git
cd reciclaje-ia-grupo4
```

Paso 2 — Crear el entorno virtual

```
# Windows
python -m venv venv
.\venv\Scripts\activate
```

```
# Linux / macOS
python3 -m venv venv
source venv/bin/activate
```

Paso 3 — Instalar dependencias

```
pip install -r requirements.txt
```

Las versiones exactas de torch (2.2.2), torchvision (0.17.2), streamlit-webrtc (0.65.4) y av (16.1.0) están fijadas en requirements.txt para garantizar reproducibilidad. No actualizar estas dependencias sin verificar compatibilidad.

Paso 4 — Verificar la instalación

```
python -c "import torch; print(torch.__version__)"  
python -c "import streamlit; print(streamlit.__version__)"
```

Paso 5 — Lanzar la aplicación

```
streamlit run app/app.py
```

La aplicación se abrirá automáticamente en el navegador en <http://localhost:8501>. Si no se abre automáticamente, navega manualmente a esa URL.

3.3 Comandos Disponibles

COMANDO	DESCRIPCIÓN
streamlit run app/app.py	Lanza la aplicación EcoRecicla IA en el navegador.
python src/train.py	Ejecuta el pipeline completo de entrenamiento (Fase 1 + Fase 2).
python src/train.py --epochs_fase1 5 --epochs_fase2 10	Entrenamiento con parámetros personalizados.
python src/evaluate.py	Evalúa el modelo sobre el conjunto de prueba y genera métricas.
python src/split_dataset.py	Genera los splits train/val/test desde data/raw/.
python fetch_trashnet.py	Descarga el dataset TrashNet desde HuggingFace.
python comprimir_fotos.py	Comprime imágenes de data/raw/ para subir a GitHub.

4. ESTRUCTURA DEL PROYECTO

El repositorio sigue la estructura de carpetas requerida por el enunciado del Proyecto 01, con módulos adicionales propios del equipo:

```

reciclaje-ia-grupo4/
├── README.md           ← Descripción, métricas y guía de instalación
├── requirements.txt     ← Dependencias con versiones exactas
├── .gitignore          ← Excluye venv/, __pycache__/, *.pth grandes
├── fetch_trashnet.py    ← Descarga TrashNet desde HuggingFace (sin Kaggle)
├── comprimir_fotos.py  ← Reduce imágenes a 800px/85% calidad para GitHub
├── data/
│   ├── raw/           ← Dataset original por clase (10 carpetas snake_case)
│   │   ├── lata/
│   │   ├── botella_pet/
│   │   ├── botella_vidrio/
│   │   ├── papel/
│   │   ├── carton/
│   │   ├── bolsa_plastica/
│   │   ├── tetrapak/
│   │   ├── organico/
│   │   ├── electronicos_pequenos/
│   │   └── no_reciclable/
│   ├── processed/     ← Splits train/val/test generados por split_dataset.py
│   └── README.md       ← Fuentes, licencias y distribución del dataset
├── notebooks/
│   ├── 01_EDA.ipynb    ← Análisis exploratorio con resultados ejecutados
│   ├── 02_train.ipynb  ← Entrenamiento con curva de aprendizaje
│   └── 03_evaluation.ipynb ← Métricas finales y matriz de confusión
├── src/
│   ├── model.py        ← CLASSES, COLOR_MAP AGG164-2021,
│   └── CLASS_THRESHOLDS
│       ├── preprocess.py ← Transformaciones: resize, normalización, augmentation
│       ├── train.py      ← Transfer Learning 2 fases con Early Stopping
│       ├── evaluate.py   ← Métricas, reporte por clase, matriz de confusión
│       ├── predict.py    ← predict_top3() y predict_frame() para video
│       └── split_dataset.py ← Split estratificado 70/15/15 semilla=42
├── app/
│   └── app.py           ← EcoRecicla IA — Interfaz Streamlit completa
├── models/
│   └── modelo_reciclaje.pth ← Modelo entrenado (MobileNetV2 fine-tuned)

```

```

├── docs/
│   ├── curva_aprendizaje.png ← Loss y accuracy por época (train vs val)
│   ├── matriz_confusion.png ← Heatmap de confusión sobre conjunto de prueba
│   ├── minutos/ ← Minutas de reuniones semanales (sem. 1-5)
│   ├── manual_tecnico.pdf
│   ├── manual_usuario.pdf
│   ├── DERCAS.pdf
│   ├── informe_final.pdf
│   └── demo_video_link.txt

```

5. DESCRIPCIÓN DE LOS MÓDULOS PRINCIPALES

5.1 src/model.py — Modelo y Mapeo

Define las constantes y funciones centrales del sistema. Contiene tres estructuras de datos clave:

CLASES — Clases del modelo

Lista ordenada alfabéticamente de las 10 clases de residuos. El orden debe coincidir exactamente con el orden en que ImageFolder de PyTorch lee las carpetas de data/processed/train/:

```

CLASES = [
    'bolsa_plastica', 'botella_pet', 'botella_vidrio', 'carton',
    'electronicos_pequenos', 'lata', 'no_reciclable',
    'organico', 'papel', 'tetrapak'
]

```

COLOR_MAP — Mapeo AGG 164-2021

Diccionario que mapea cada clase al recipiente oficial según el Acuerdo Gubernativo 164-2021:

```

COLOR_MAP = {
    'lata': ('Metal', ' ☐ Blanco', 'Enjuagar y aplastar'),
    'botella_pet': ('Plástico', ' ☐ Blanco', 'Quitar tapón y aplastar'),
    'botella_vidrio': ('Vidrio', ' ☐ Blanco', 'No romper, depositar entero'),
    'papel': ('Papel', ' ☐ Blanco', 'Mantener seco y limpio'),
    'carton': ('Papel', ' ☐ Blanco', 'Doblar y atar si es grande'),
}

```

```

'bolsa_plastica': ('Plástico', '  Blanco', 'Vaciar completamente'),
'tetrapak': ('Mixto', '  Blanco', 'Enjuagar y aplastar'),
'organico': ('Orgánico', '  Verde', 'Para compostaje o biogás'),
'electronicos_pequenos': ('Electrónico', '  Negro', 'Llevar a punto limpio'),
'no_reciclable': ('Otros', '  Negro', 'Depositar en basura general'),
}

```

CLASS_THRESHOLDS — Umbrales adaptativos

Define el umbral de confianza mínimo por clase. Clases con menos datos de entrenamiento tienen umbrales más bajos para evitar falsas alertas de baja confianza:

```

CLASS_THRESHOLDS = {
    'bolsa_plastica': 0.45, # Pocos datos — umbral bajo
    'tetrapak': 0.50,
    'no_reciclable': 0.50,
    'botella_vidrio': 0.55,
    'organico': 0.55,
    'electronicos_pequenos': 0.60,
    'lata': 0.65,
    'carton': 0.65,
    'papel': 0.65,
    'botella_pet': 0.70, # Muchos datos — umbral estándar
}

```

5.2 src/preprocess.py — Preprocesamiento

Define tres pipelines de transformaciones de imagen según el contexto de uso:

FUNCIÓN	USO	TRANSFORMACIONES
get_train_transforms()	Entrenamiento	Resize(256), RandomCrop(224), RandomHorizontalFlip, RandomRotation($\pm 15^\circ$), ColorJitter, GaussianBlur, Normalize ImageNet.
get_val_transforms()	Validación / Prueba	Resize(224), ToTensor, Normalize ImageNet. Sin augmentation.

get_inference_transforms()	Inferencia en app	Resize(224), ToTensor, Normalize ImageNet. Idéntico a val para consistencia.
----------------------------	-------------------	--

Los valores de normalización [0.485, 0.456, 0.406] (media) y [0.229, 0.224, 0.225] (std) corresponden a las estadísticas del dataset ImageNet. Deben mantenerse idénticos en entrenamiento e inferencia para que el modelo funcione correctamente.

5.3 src/train.py — Entrenamiento

Implementa el pipeline completo de Transfer Learning en dos fases con Early Stopping:

FASE	PARÁMETROS	DESCRIPCIÓN
Fase 1 — Clasificador	5 épocas · lr=1e-3	Todas las capas base de MobileNetV2 congeladas. Solo se entrena la capa clasificadora final (Linear 1280→10). Adam optimizer + ReduceLROnPlateau.
Fase 2 — Fine-tuning	10 épocas · lr=1e-4	Se descongelan las últimas 30 capas del modelo base. Continúa el entrenamiento con learning rate menor para ajuste fino. Early Stopping con patience=5.

Parámetros del script train.py:

```
python src/train.py --data_dir data/processed --epochs_fase1 5 --epochs_fase2 10
```

El modelo se guarda automáticamente en models/modelo_reciclaje.pth cada vez que el F1-macro de validación mejora. La curva de aprendizaje se exporta a docs/curva_aprendizaje.png al finalizar.

5.4 src/evaluate.py — Evaluación

Ejecuta la evaluación completa del modelo sobre el conjunto de prueba y genera:

- Reporte de clasificación por clase: Precisión, Recall, F1-score y Soporte.
- Métricas globales: Accuracy, Top-2 Accuracy, F1-Macro y F1-Weighted.
- Matriz de confusión visual en docs/matriz_confusion.png mediante seaborn heatmap.
- Identificación automática de los 3 errores de confusión más frecuentes.

```
python src/evaluate.py
```

5.5 src/predict.py — Inferencia

Define dos funciones de predicción optimizadas para sus respectivos contextos:

FUNCIÓN	DESCRIPCIÓN
<code>predict_top3(image, model)</code>	Usada en modo foto y archivo. Retorna las 3 clases más probables con sus confianzas, categoría, color de recipiente y consejo. Permite implementar el umbral de duda comparando <code>top[0]</code> vs <code>top[1]</code> .
<code>predict_frame(frame, model)</code>	Optimizada para video en tiempo real. Recibe un frame numpy (BGR), lo convierte a PIL RGB, aplica transformaciones en memoria y retorna top-2 para poder construir el mensaje de duda sin penalizar el rendimiento.

5.6 Interfaz Interna de Inferencia (API Interna)

Aunque el sistema no expone una API REST pública, cuenta con una **interfaz interna** bien definida entre la capa de presentación (`app.py`) y el modelo. Esta interfaz es la que permite la comunicación entre módulos:

Funciones de Inferencia

FUNCIÓN	PARÁMETROS	RETORNO	USO
<code>predict_top3(image, model)</code>	<code>image:</code> <code>PIL.Image</code> <code>model:</code> <code>torch.nn.Module</code>	Lista de 3 predicciones: • <code>class:</code> str • <code>confidence:</code> float • <code>color:</code> str • <code>tip:</code> str	Clasificación en modo foto y archivo
<code>predict_frame(image, model)</code>	<code>image:</code> <code>PIL.Image</code> <code>model:</code> <code>torch.nn.Module</code>	Predicción optimizada: • <code>class:</code> str • <code>confidence:</code> float • <code>color:</code> str • <code>emoji:</code> str • <code>tip:</code> str	Video en tiempo real
<code>load_model(model_path, device)</code>	<code>model_path:</code> str <code>device:</code> str (cpu / cuda)	<code>torch.nn.Module</code>	Carga única del modelo al iniciar

Ejemplo de uso interno (extraído de app.py)

```
# Carga del modelo (una sola vez, con caché)
@st.cache_resource
def load_cached_model():
    return load_model('models/modelo_reciclaje.pth', 'cpu')

# Predicción en modo foto
image = Image.open(captured_file)
top3 = predict_top3(image, model)

# Verificación de umbral de duda
if top3[0]['confidence'] < CLASS_THRESHOLDS[top3[0]['class']]:
    st.warning(f" 😬 Dudoso: Parece {top3[0]['class']}, pero podría ser {top3[1]['class']}")
else:
    st.success(f" ✅ {top3[0]['class']} — {top3[0]['color']}")
```

Esta arquitectura interna permite **cambiar el modelo sin modificar la interfaz y agregar nuevas funcionalidades** sin afectar la lógica de predicción.

6. DATASET Y FLUJO DE DATOS

6.1 Fuentes del Dataset

FUENTE	IMÁGENES	CLASES	ACCESO
TrashNet	~2,527	6 (metal, plastic, glass, paper, cardboard, trash)	GitHub — garythung/trashnet o HuggingFace (fetch_trashnet.py)
Garbage Classification	~2,500	6 categorías similares a TrashNet	Kaggle: asdasdasdas/garbage-classification
Fotos propias equipo	1,400+	Las 10 clases del modelo	Recolección manual con teléfonos móviles — contexto guatemalteco

6.2 Pipeline de Procesamiento de Datos

El flujo completo de datos desde la recolección hasta el entrenamiento sigue esta secuencia:

- **Recolección:** imágenes descargadas de fuentes públicas + fotos tomadas con teléfonos móviles por el equipo.

- Organización: carpetas en data/raw/ por clase, nombres en snake_case sin tildes.
- Compresión: script comprimir_fotos.py reduce imágenes a 800px máximo y 85% calidad JPEG para compatibilidad con GitHub (de ~14 MB a ~0.3 MB por foto).
- EDA: análisis en 01_EDA.ipynb — distribución de clases, muestras visuales, tamaño promedio y detección de imágenes borrosas (varianza del Laplaciano < 100).
- Split: src/split_dataset.py genera data/processed/ con splits estratificados 70/15/15 usando semilla=42.
- Entrenamiento: src/train.py consume data/processed/train/ y data/processed/val/ con augmentation en el conjunto de entrenamiento.
- Evaluación: src/evaluate.py evalúa sobre data/processed/test/ — 208 imágenes nunca vistas durante el entrenamiento.

6.3 Distribución del Dataset

El dataset final contiene más de 1,900 imágenes distribuidas en las 10 clases. Las clases con mayor volumen provienen de los datasets públicos (papel, botella_pet) y las clases con menor volumen dependen principalmente de fotos propias del equipo (tetrapak, electronicos_pequenos):

Las clases con menor volumen de imágenes (tetrapak, bolsa_plastica, electronicos_pequenos, no_reciclable) presentan los menores F1-scores en evaluación. Ampliar estas clases tiene el mayor potencial de mejora sobre el rendimiento global del modelo.

7. INTERFAZ DE USUARIO — APP/APP.PY

7.1 Descripción General

La interfaz EcoRecicla IA está construida con Streamlit y expone los tres modos de clasificación en una sola página web accesible desde cualquier navegador moderno. Al iniciar la aplicación, el modelo se carga una única vez en memoria mediante `@st.cache_resource` para minimizar la latencia en cada interacción.

7.2 Modo Foto Instantánea

Utiliza el componente `st.camera_input` de Streamlit para capturar una foto desde la cámara web. La imagen se procesa con `predict_top3()` y el resultado se renderiza con la clase, confianza, recipiente AGG 164-2021 y consejo. Si la confianza es menor al umbral definido en `CLASS_THRESHOLDS` para esa clase, se muestra el mensaje de duda.

7.3 Modo Video en Tiempo Real

Utiliza `streamlit-webrtc` para acceder a la cámara vía WebRTC directamente en el navegador. Un callback de video procesa cada frame con `predict_frame()` y superpone el resultado como overlay sobre el video en vivo. El modo video requiere conexión a internet estable para el protocolo WebRTC.

Si el modo video no activa la cámara, verificar que el navegador tiene permisos de cámara habilitados y que la conexión a internet es estable. Chrome es el navegador más compatible con `streamlit-webrtc`.

7.4 Modo Archivo

Permite cargar archivos de imagen (JPG, PNG) o video (MP4, AVI, MOV). Para imágenes, se aplica `predict_top3()` directamente. Para videos, se muestrean fotogramas distribuidos a lo largo del archivo, se clasifican individualmente con `predict_frame()` y la clase ganadora se determina por moda (Counter). Si la confianza promedio es baja, se extrae la segunda clase más frecuente para el mensaje de duda.

7.5 Umbral de Duda

El umbral de duda es una funcionalidad que hace al sistema más honesto cuando el modelo no está seguro de su predicción. El comportamiento es el siguiente:

- Si confianza $\geq$ umbral de la clase: se muestra el resultado normal con clase, recipiente y consejo.
- Si confianza $<$ umbral de la clase: se muestra el mensaje ' 🤔 Dudoso — Parece [clase X], pero podría ser [clase Y]. ¿Puedes acercar más la cámara o mejorar la iluminación?' con las dos clases más probables.
- Los umbrales varían por clase (45%-70%) según el volumen de datos de entrenamiento disponible.
-

8. DEPENDENCIAS Y VERSIONES

El archivo `requirements.txt` fija las versiones exactas de todas las dependencias para garantizar reproducibilidad del entorno:

LIBRERÍA	VERSIÓN	USO EN EL PROYECTO
torch	2.2.2	Framework de deep learning — entrenamiento e inferencia de MobileNetV2.
torchvision	0.17.2	Modelos preentrenados, transformaciones y datasets de imágenes.
streamlit	1.33.0	Framework web para la interfaz interactiva EcoRecicla IA.
streamlit-webrtc	0.65.4	Acceso a cámara web vía WebRTC para el modo video en tiempo real.
av	16.1.0	Procesamiento de video requerido por streamlit-webrtc.
Pillow	10.3.0	Carga y manipulación de imágenes PIL.
numpy	1.26.4	Operaciones matriciales y manejo de arrays de píxeles.
scikit-learn	1.4.2	F1-macro, matriz de confusión y métricas por clase.
matplotlib	3.8.4	Gráficas de curva de aprendizaje.
seaborn	0.13.2	Heatmap visual de la matriz de confusión.
opencv-python	4.9.0.80	Preprocesamiento de imágenes y conversión de frames de video.
albumentations	1.4.3	Data augmentation avanzado durante el entrenamiento.

9. SEGURIDAD, PRIVACIDAD Y MANTENIMIENTO

9.1 Privacidad por Diseño

- **Cero persistencias:** El sistema no almacena ninguna imagen, video o dato personal. Todo el procesamiento ocurre en RAM y se descarta al cerrar la sesión.
- **Sin logs:** No se registra historial de clasificaciones ni metadatos de las imágenes procesadas.
- **Sin autenticación:** No se requiere registro ni inicio de sesión, eliminando riesgos de filtración de credenciales.
- **Procesamiento local:** El modo video WebRTC procesa frames directamente en el navegador; nunca se transmiten al servidor.

9.2 Limitaciones de Seguridad

ASPECTO	ESTADO ACTUAL	RECOMENDACIÓN PARA PRODUCCIÓN
Cifrado de comunicaciones	No aplica (ejecución local)	Implementar HTTPS + reverse proxy (Nginx)
Autenticación	No requerida	Agregar autenticación básica si el sistema se despliega públicamente
Control de acceso	No implementado	Definir roles si existen múltiples usuarios
Sanitización de entradas	Validación mediante PIL	Mantener validaciones al agregar nuevos formatos

9.3 Mantenimiento

- Para agregar una nueva clase de residuo: agregar la carpeta en data/raw/, agregar la entrada en CLASSES, COLOR_MAP y CLASS_THRESHOLDS en src/model.py, recolectar mínimo 50 imágenes propias, ejecutar split_dataset.py y reentrenar con train.py.
- Para actualizar dependencias: modificar versiones en requirements.txt, verificar compatibilidad entre torch y torchvision, y ejecutar las pruebas con evaluate.py para confirmar que el modelo sigue funcionando.
- El modelo .pth debe versionarse junto con el código. Si supera 25 MB, almacenar en Google Drive y actualizar el enlace en README.md.

9.3.1 Validación del modelo después de cambios

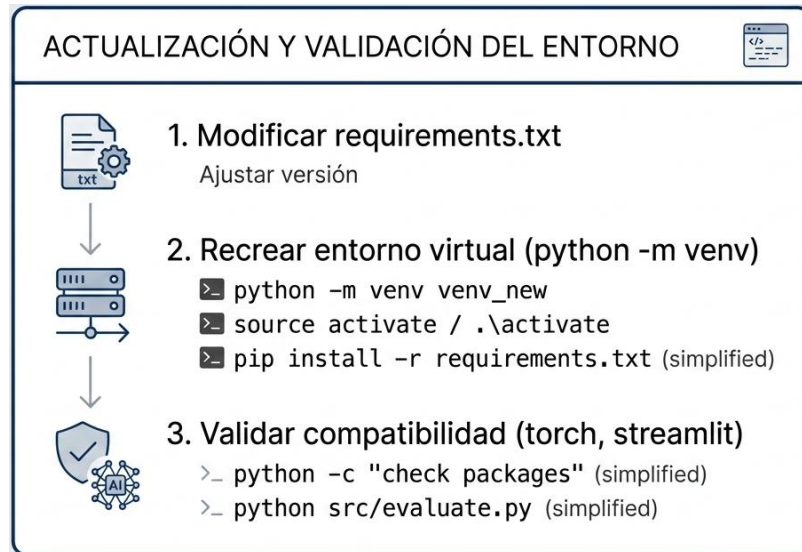
Métrica	Umbral mínimo	Propósito
Accuracy Global	$\geq 80\%$	Precisión general del modelo
F1-Macro	$\geq 75\%$	Balance entre clases
F1-Weighted	$\geq 78\%$	Rendimiento ponderado según distribución

9.3.2 Control de versiones del modelo

El archivo models/modelo_reciclaje.pth sigue esta convención:

Acción	Procedimiento
Reentrenamiento exitoso	git commit -m "train: reentrenamiento vX - accuracy YY.YY% - F1-macro ZZ.ZZ%"
Modelo > 25 MB	Subir a Google Drive y actualizar enlace en README.md
Rollback necesario	Recuperar versión anterior desde historial de commits

9.3.3 Actualización de dependencias



Dependencias críticas que NO deben actualizarse sin pruebas: torch, torchvision, streamlit-webrtc, av.

9.4 Resolución de Problemas Comunes (Troubleshooting)

Problema	Síntoma	Solución
Modelo no carga	FileNotFoundError: modelo_reciclaje.pth	Verificar que el archivo existe en models/ o reentrenar con python src/train.py
Out of Memory	CUDA out of memory	Forzar CPU: device='cpu' en load_model()
WebRTC no inicia	Modo video no activa cámara	Usar Chrome; verificar permisos de cámara; conexión a internet estable

Baja precisión	Predicciones incorrectas frecuentes	Recolectar más imágenes de clases con bajo F1 y reentrenar
Transformaciones fallan	PIL.UnidentifiedImageError	El archivo no es una imagen válida; verificar formato JPG/PNG
Streamlit no arranca	ModuleNotFoundError	Activar entorno virtual: venv\Scripts\activate y pip install -r requirements.txt

9.5 Procedimiento de Validación Completa (Checklist)

Antes de considerar una versión del sistema como "estable", ejecutar:

- python src/split_dataset.py (si se agregaron imágenes nuevas)
- python src/train.py (entrenamiento completo)
- python src/evaluate.py (métricas $\geq$ umbrales definidos)
- streamlit run app/app.py (prueba manual de los 3 modos)
- Verificar que el mensaje de duda aparece cuando corresponde
- Confirmar que los colores de recipiente coinciden con AGG 164-2021
- git commit con el nuevo modelo y métricas en el mensaje

10. FUTURAS MEJORAS Y ESCALABILIDAD

El sistema fue diseñado de forma modular para facilitar la incorporación de mejoras futuras:

MEJORA	DESCRIPCIÓN TÉCNICA
Detección múltiple (YOLOv8)	Reemplazar la clasificación por imagen completa con detección de objetos para clasificar varios residuos en una sola imagen con bounding boxes.
API REST (FastAPI)	Exponer el modelo como endpoint REST para integración con apps móviles. Retornar JSON con clase, confianza y recomendación.
Reentrenamiento continuo	Implementar un pipeline de reentrenamiento automático cuando se agreguen nuevas imágenes al repositorio, usando GitHub Actions o similar.
Soporte multilingüe	Agregar soporte para idiomas mayas (k'iche', q'eqchi') en los mensajes de la interfaz mediante i18n con diccionarios Python.
Dashboard de impacto	Contador de objetos clasificados en la sesión con estimación de kg correctamente separados — útil para demostraciones en Expoferia.

Segmentación con U-Net	Aislar el área del residuo del fondo antes de clasificar, reduciendo errores causados por fondos complejos o iluminación irregular.
-------------------------------	---

El sistema EcoRecicla IA está diseñado para ejecutarse localmente, pero puede desplegarse en plataformas cloud gratuitas sin modificar el código. Esta sección documenta dos opciones compatibles.

10.1 Opción A — Streamlit Community Cloud

Requisitos previos:

- Repositorio público en GitHub con el código completo
- Archivo `models/modelo\_reciclaje.pth` accesible (Git LFS o Google Drive)

Pasos:

1. Crear nuevo Space → elegir SDK: **Streamlit**
2. Subir archivos vía interfaz web o Git:
 - `app/app.py`
 - `src/` (carpeta completa)
 - `models/modelo\_reciclaje.pth`
 - `requirements.txt`
3. El Space compilará automáticamente y generará URL tipo:
`https://huggingface.co/spaces/usuario/ecoreciclaia`

10.3 Comparativa de Opciones

10.4
sobre

CARACTERÍSTICA	STREAMLIT CLOUD	HUGGING FACE SPACES
Costo	Gratuito	Gratuito
Límite de almacenamiento	1 GB	50 GB
Soporte WebRTC	Sí	Sí
Modelo > 25 MB	Requiere Google Drive	Subida directa
Inactividad	Suspensión tras 48 h	Suspensión tras 48 h
Configuración	Mínima	Mínima

Nota

despliegue actual

Al momento de publicación de este manual, **el sistema no está desplegado en la nube**. Se ejecuta exclusivamente en entorno local mediante:

```
streamlit run app/app.py
```

Las secciones 10.1 a 10.3 documentan opciones futuras para publicación académica o demostraciones.

11. FUTURAS MEJORAS Y ESCALABILIDAD (AMPLIADO)

El sistema fue diseñado de forma modular para facilitar la incorporación de mejoras. A continuación, se detalla el impacto técnico de cada posible mejora:

11.1 Mejoras Priorizadas:

MEJORA	IMPACTO	DIFICULTAD	PASOS TÉCNICOS
API REST (FastAPI)	Integración con apps móviles	Media	1. Crear api.py con endpoints 2. Mover lógica de predict.py 3. Retornar JSON
Detección múltiple (YOLOv8)	Clasificar varios objetos por imagen	Alta	1. Anotar bounding boxes 2. Entrenar detector 3. Integrar con clasificador
Reentrenamiento automático	Actualización continua del modelo	Media	1. Configurar GitHub Actions 2. Ejecutar train.py en runner 3. Subir modelo a Hugging Face
Soporte multilingüe	K'iche', Q'eqchi'	Baja	1. Crear diccionarios de traducción 2. Detectar idioma del navegador 3. Renderizar mensajes dinámicos
Dashboard de impacto	Mostrar kg de residuos separados	Baja	1. Almacenar clasificaciones en sesión 2. Acumular contadores por clase 3. Estimar peso por tipo de residuo

CONCLUSIÓN

EcoRecicla IA es un sistema técnicamente sólido, construido sobre principios de ingeniería de software bien establecidos: modularización del código, reproducibilidad del entorno, versionamiento con Git y documentación técnica completa.

La arquitectura en tres capas (datos, modelo, presentación) facilita el mantenimiento independiente de cada componente. La implementación de Transfer Learning sobre MobileNetV2 demostró ser la elección correcta para el alcance del proyecto: alto rendimiento con recursos limitados, entrenamiento viable en CPU y despliegue sin dependencias de hardware especial.

Las decisiones técnicas más relevantes del proyecto — umbral de duda adaptativo por clase, migración a WebRTC, corrección de colores según AGG 164-2021 y pesos por clase en la función de pérdida — responden directamente a observaciones reales del catedrático y limitaciones identificadas durante el desarrollo, lo que demuestra un proceso de desarrollo iterativo y orientado a la calidad.